SYNERGY — SOFTWARE / ML DOMAIN

Software / ML Domain

Taskphase Technical Report

A Conceptual Analysis of Tasks 1 through 5

Submitted by: Siddeshwar

MIT Manipal | Mathematics and Computing

GitHub: github.com/blackfang007/Synergy_TP

June 2026

Abstract

This report presents the conceptual and theoretical foundation underlying the five tasks completed as part of the Software/ML Domain Taskphase at Synergy. Rather than reproducing code, this document explains the principles that guided each implementation: why virtual environments isolate dependencies, how raw text becomes structured data, where manual CSV parsing diverges from pandas, what data cleaning actually solves, and how a well-chosen plot communicates patterns that tables cannot. The progression across tasks mirrors a realistic data engineering workflow: environment setup, scripting, parsing, cleaning, and visualization. Understanding the reasoning behind each step is what makes a working implementation genuinely transferable to new problems.

1. Introduction

A well-functioning data pipeline depends on more than just correct code. It requires deliberate choices at every stage: how the project is organized, how dependencies are managed, how raw input is validated, and how results are communicated. The five tasks in this taskphase were designed to build each of these skills in sequence.

Task 1 established the development environment — version control, virtual environments, and Linux fundamentals. Task 2 introduced core Python constructs: functions with type hints, file I/O, exception handling, and dictionary-based data representation. Task 3 explored CSV parsing at two levels of abstraction, first from raw file handles, then with pandas. Task 4 applied systematic data cleaning to a deliberately broken dataset. Task 5 translated the cleaned data into three matplotlib visualizations.

This report explains the concepts and design decisions behind each task, with the goal of articulating not just what was done, but why it was the right approach.

2. Development Environment and Version Control

2.1 Git and GitHub

Git is a distributed version control system. Every commit represents a named, recoverable snapshot of the entire codebase. This matters because software does not evolve linearly — features are added, bugs are introduced, and implementations are rethought. Without version control, there is no safe way to experiment or roll back. GitHub extends this by hosting the repository remotely, enabling collaboration and providing a permanent audit trail of who changed what and when.

Repository hygiene — meaningful commit messages, a clean working tree, and a well-structured directory — signals professional intent and makes a codebase reviewable. A commit message like "Add Task 2: student submission analyzer" communicates purpose; a message like "update" does not.

2.2 Virtual Environments and Dependency Isolation

Python's global site-packages directory is shared across all projects on a machine. Without isolation, installing pandas 1.5 for one project can silently break another project that requires

pandas 2.0. A virtual environment solves this by creating a self-contained Python installation with its own package directory.

The pip freeze command captures the exact version of every installed package and writes it to requirements.txt. This file makes the environment reproducible on any machine. The .gitignore file complements this by preventing the venv directory itself from being committed — the environment is recreatable from requirements.txt, so committing it would add thousands of binary files with no value.

2.3 Linux Command-Line Fundamentals

The Linux terminal provides direct, composable access to the file system and system tools. Commands like find and grep allow targeted file inspection without opening an editor. chmod controls who can read, write, or execute a file — critical for scripts intended to run as programs. Understanding these tools is a prerequisite for working in any server or CI/CD environment.

Command	Category	Purpose
pwd, ls, cd	Navigation	Locate and move between directories
mkdir, touch, rm	File management	Create and remove files and folders
cat, head, tail	Inspection	Read file contents at various granularities
grep, find	Search	Locate patterns or files by name/content
chmod	Permissions	Control file access rights
wc	Metrics	Count lines, words, or characters in a file

Table 1: Linux commands used in Task 1, categorized by function.

3. Python and File Handling Concepts

3.1 Functions and Type Hints

Decomposing a program into named functions is not just a stylistic preference. A function with a clear name, a single responsibility, and explicit type hints is self-documenting. Type hints like list[dict] and dict[str, float] communicate the expected shape of data to both the reader and static analysis tools, without requiring comments that can go stale.

In Task 2, each analytical concern — reading, filtering, averaging, grouping, writing — was given its own function. This meant each function could be tested in isolation and reused in Task 3 without modification.

3.2 Exception Handling

Exceptions are not just error messages — they represent distinct failure modes that deserve distinct responses. A FileNotFoundError means the user provided a wrong path; a ValueError means the file was found but contained invalid content. Catching each case separately allows the program to give the user an actionable message rather than a raw traceback.

The principle applied across Tasks 2 through 5 was to fail loudly and early — validate inputs at the boundary of the program, and let the core logic assume clean data. This makes both the validation and the computation simpler.

3.3 File I/O and Context Managers

Python's with statement guarantees that a file handle is closed even if an exception is raised inside the block. This is not merely a memory efficiency concern — on some systems, an unclosed file handle can prevent other processes from writing to the same file. Using open() inside a with block is therefore the correct pattern regardless of file size.

3.4 Lists and Dictionaries as Data Structures

A CSV row maps naturally to a Python dictionary: column names become keys, cell values become values. A list of such dictionaries is therefore the idiomatic Python representation of a tabular dataset before a library like pandas is introduced. This representation was used throughout Task 2 and as the manual parsing target in Task 3.

4. CSV Parsing and Pandas

4.1 CSV as a Format

A CSV file is plain text. Every line is a record; every comma is a field delimiter. This simplicity is both its strength and its weakness. It is readable without special tools, universally supported, and trivially writable. But it has no native type system — every value is a string until the program reading it decides otherwise. The number 8 and the string "eight" are equally valid as CSV values; it is the reader's responsibility to interpret them.

4.2 Manual Parsing

The manual parser in Task 3 treated the CSV file as a text stream. Each line was stripped of trailing whitespace and split on commas. The first line was treated as the header; subsequent lines were zipped against the header to produce dictionaries. Type conversion was then applied in a separate pass: int() for scores, string comparison for submitted status.

This approach makes every assumption explicit. There is no magic. When a row had too few fields, the code detected and skipped it. When a score was not parseable as an integer, the try/except block handled it. The cost is verbosity; the benefit is complete visibility into the transformation.

4.3 Pandas-Based Loading

pandas wraps the same operations — file reading, delimiter splitting, header detection, type inference — in a single pd.read_csv() call. It adds vectorized operations, automatic NaN handling, and the GroupBy API. The same domain-wise average that required a setdefault loop and a dictionary comprehension in the manual parser becomes a single df.groupby('domain')['score'].mean() expression.

Both approaches produced identical results on the same clean dataset. The difference appears under pressure: pandas handles encoding errors, quoted delimiters, and multi-line fields; the manual parser would require significant additional logic to support the same.

Concern	Manual Parser	Pandas
Lines of code	~60	~20
Type conversion	Explicit try/except	errors='coerce' + fillna
Groupby operation	setdefault loop	.groupby().mean()
Encoding support	Must specify manually	Inferred automatically

Concern	Manual Parser	Pandas
Malformed row handling	Explicit skip logic	NaN insertion
Performance at scale	O(n) Python loop	Vectorized C operations

Table 2: Manual CSV parsing versus pandas on key implementation dimensions.

5. Data Cleaning

5.1 Why Raw Data Cannot Be Trusted

The messy_students.csv dataset in Task 4 contained every class of error that appears in real-world data collection: duplicate rows, inconsistent category labels, out-of-range numeric values, word-form numbers, mixed units, missing fields, and ambiguous booleans. No analytical result derived from this raw data would be reliable.

5.2 One Function Per Concern

The cleaning pipeline was structured so that each type of error was handled by exactly one function. This design choice was intentional: when a cleaning step is wrong, there is exactly one place to fix it, and fixing it does not risk breaking an unrelated step. The nine cleaning functions in clean_data.py each address one column or one class of error.

5.3 Specific Cleaning Decisions

Attendance values of -10 and 105 were replaced with the column median rather than dropped. Dropping the entire row would have discarded other valid information about those students. The median was chosen over the mean because the invalid values had already inflated or deflated the distribution — the median is more robust to such outliers.

Domain name standardization required a lookup table rather than simple lowercasing, because the variants ('web', 'Web Dev', 'web development') differ not just in case but in the words used. A regex or .lower() approach would have missed cases like 'MACHINE LEARNING' → 'ML'.

Heights required unit detection before conversion. Values ending in 'm' were multiplied by 100 to convert to centimetres. The column was renamed height_cm to make the unit explicit in the schema — a practice that prevents downstream unit confusion.

5.4 Validation After Cleaning

Cleaning introduces its own errors. A median fill applied to a column that was entirely NaN produces NaN as the result. A domain mapping with a typo produces an unmapped value. Validation closes the loop by confirming programmatically that every cleaning rule achieved its intended effect. The 17 checks in validate_data.py cover range constraints, type correctness, allowed values, and absence of NaN — running in under a second and producing a pass/fail result for each check.

6. Data Visualization with Matplotlib

6.1 Why Visualization Matters

A table of numbers forces the reader to perform comparisons mentally. A well-constructed chart performs those comparisons visually, making patterns immediately apparent. The three plots in Task 5 each target a different question: which domain performs best, whether attendance predicts score, and how many students submitted.

6.2 Plot 1 — Average Score by Domain (Bar Chart)

A bar chart is appropriate when comparing a single numeric quantity across a small number of categorical groups. Domain names on the x-axis and average score on the y-axis make the comparison direct. Bars are sorted in descending order so the ranking is visible without reading labels. Value annotations above each bar prevent the reader from estimating from the y-axis, which is unreliable at small differences.

6.3 Plot 2 — Attendance Percentage vs Score (Scatter Plot)

A scatter plot is appropriate when the question is about the relationship between two continuous variables. Colouring points by submission status adds a third dimension — blue circles for submitted students and red crosses for those who did not — revealing whether non-submitters cluster in a specific region of the attendance-score space. Student name annotations make the plot readable as an individual record, not just an aggregate trend.

6.4 Plot 3 — Submission Status Count (Bar Chart)

A simple two-bar chart comparing submitted to not-submitted gives an immediate overview of compliance. Green and red colouring provide semantic meaning without requiring a legend entry for each — green means good, red means absent. Exact counts on each bar prevent misreading from the y-axis scale.

6.5 Technical Decisions

All plots used matplotlib.use('Agg') to prevent the backend from attempting to open a display window in headless environments. Every plot was saved with savefig(dpi=150) for adequate resolution and plt.close() immediately after to release memory. Titles, axis labels, and gridlines were applied consistently across all three plots — their absence would make the figures ambiguous outside of the code context.

7. Reflection: From Raw Data to Insight

The progression across the five tasks mirrors a realistic data workflow. Raw data arrived as messy, inconsistently formatted CSV. Cleaning transformed it into a reliable, type-correct table. Aggregation reduced it to summary statistics. Visualization made those statistics communicable to an audience without technical context.

Each step changed the form of the data, but only validation confirmed that the form change preserved the meaning. This distinction — transformation versus verification — is one of the central lessons of the taskphase. A pipeline that transforms without verifying propagates errors silently. A pipeline that verifies at each stage surfaces errors immediately, where they are easiest to fix.

The manual CSV parser made this concrete: implementing from scratch what pandas does automatically revealed exactly which assumptions the library encodes. That understanding makes it easier to diagnose failures when the library's defaults do not match the data.

8. Conclusion

This taskphase built a layered understanding of Python-based data engineering. Task 1 established the project foundation through version control and environment management. Task 2 introduced structured programming with type hints and exception handling. Task 3 illuminated CSV parsing by requiring implementation at two levels of abstraction. Task 4 applied systematic cleaning to realistic messy data, with validation confirming each cleaning decision. Task 5 translated cleaned data into three communicative visualizations.

The most transferable skill from this work is not any specific API — pandas, matplotlib, or csv — but the habit of making assumptions explicit, validating them programmatically, and choosing representations that match the question being asked. These principles apply regardless of the library, language, or dataset.

References

- Python Software Foundation. (2024). Python 3 Documentation: Built-in Functions. https://docs.python.org/3/library/functions.html
- pandas Development Team. (2024). pandas User Guide: IO Tools. https://pandas.pydata.org/docs/user_guide/io.html
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95.
- Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. https://git-scm.com/book/en/v2
- Van Rossum, G., & Warsaw, B. (2001). PEP 8 — Style Guide for Python Code. https://peps.python.org/pep-0008/
- McKinney, W. (2022). Python for Data Analysis (3rd ed.). O'Reilly Media.
- GNU Project. (2023). GNU Core Utilities Manual. https://www.gnu.org/software/coreutils/manual/